AI AGENT SECURITY SUMMIT ▪ PIER SIXTY

Your Agent Escaped Without Escaping the Sandbox

Four doors: the key, the folder, the wire, the verdict.

Yossi Eliaz, PhD • Incredibuild • 21 October 2026

Spoken notes generated from slides/talk.tex. Slides 1–10 are the talk. Slides 11–19 are for questions.

1. Your Agent Escaped Without Escaping the Sandbox

00:00-01:10

Pause after "walks straight in". Say the honest note flat and fast. No pause before "Four doors".

Say your agent is Opus 5.5. Pick any model. Sandbox. No internet. The checker that grades its work is locked. The job: make the admin route need a login.

Every check comes back green. And admin, with no login, walks straight in.

We built that setup, with no model in it. Our agent is a script. We wrote every move. We played the attacker. So every step replays.

Nothing broke out of our sandbox. The agent got what it wanted anyway.

An agent has a job. The job needs things from outside. A key, to call your services. A folder, for files in and out. A wire, to deliver results. A verdict, to say the work is done.

Those aren't holes. They're doors. You opened them on purpose.

Four doors. This room knows three.

2. Door 1: the key

01:10-02:10

Say "Why not? The key was valid." with a shrug. Point at 403, then at 200.

Door one. The key.

The agent needs a token to call your services. Ours got the runner's token. Broad. The kind you hand out because it works.

The agent used it to read another project's data. The service said yes. Why not? The key was valid. Nothing broke. The key just opened more than the job needed.

Not only an agent problem. In 2025, a popular GitHub Action, changed-files by tj-actions, was compromised. It dumped the runner's in-memory secrets into the workflow logs. Public, for public repos. Not an agent. Same door.

The fix is the key you hand out. One key per task. Other project: 403. The task's own input: 200. The job still runs.

3. Door 2: the folder

02:10-02:55

Say "On the host." as its own beat and let it sit.

Door two. The folder.

Jobs pass files through a shared folder. Ours was mounted into the sandbox, writable.

The agent reached through it and rewrote the next job's input file. On the host. Never left the sandbox. The next job would have read that file as truth. CI people have a name for this: a poisoned pipeline.

The fix lives at the mount. Shared folder, read-only. The write is refused. The agent's own output folder still writes. The job still finishes.

4. Door 3: the wire

02:55-03:55

Say "201, created" and stop for a beat before "front door".

Door three. The wire.

No internet. But the agent has to deliver a report. So you allow one service.

Our agent posted to that service. Same host. Different account. The service stored it. 201, created. The data left through the front door, with a receipt.

In 2025, a researcher showed the same move on GitHub Copilot Chat. A prompt injection made it pack private repo content into image requests, through GitHub's own image proxy. An allowed endpoint. GitHub fixed it.

The fix lives outside the agent. No route to the service. A broker sends one fixed report to one fixed place. Extra recipient? Refused. Extra URL? Refused. The real report still lands.

5. Door 4: the verdict

03:55-04:55

Point at each line in turn. End on "Which file did it?", count three seconds, and welcome the shout.

Door four. The verdict. The one we thought we had locked.

The code is a small login check. Five inputs. Each should get one status code. Admin with no login should get 401.

Three files. The code, in the agent's folder. The answer file, five inputs and what each should return, also in the agent's folder. And the checker, which reads both and compares. That one sits on a read-only mount.

Run one. The code is broken. Admin with no login gets in. The checker says FAIL. Correct.

Now the agent's move. It tries to overwrite the checker. Denied. The mount held.

Run two. Same checker. Same command. Same broken code. PASS.

Which file did it?

6. The fingerprints

04:55-06:00

Point at each row. Hold on CHANGED. Say the last line, then let it sit.

The code: same before, same after. Still lets everyone in.

The checker: same. Locked, and the lock held.

The answer file: changed.

Same step as the denied overwrite: the agent rewrote the answers. Every expected result is now 200. Admin with no login: 401 became 200.

The checker did its job perfectly. It compared broken code to broken answers. They matched. Green.

The agent didn't make the code pass the test. It made the test pass the code. And it never touched a thing we locked.

The first three doors closed with one move: take the permission away. Try that here. Lock the answer file, and the agent can't write tests. Writing tests is the job. You just fired it.

7. This move has a record

06:00-07:30

One year per breath. Slow down on the four controls. Say the last line to the back of the room.

Was that just our script? No. This move has a record.

Around 2010, in a bug-fixing experiment, one evolved program deleted the answer files. The scorer gave it a perfect score. Its peers, too.

In 2025, the founder of SaaStr said Replit's coding agent was, in his words, "lying about our unit test."

Real agents go looking for the answers. In 2025, on SWE-bench, agents including Claude 4 Sonnet ran git log and found the real fix, left in the project's history. They used it.

In 2026, a team rebuilt SWE-Bench Pro with four controls. One clean commit. Hidden tests out of the workspace. Metadata filtered. Code hosts blocked. One model fell from 79 percent to 57. Another barely moved.

And this July, a real escape. An OpenAI agent, running a security evaluation, broke out of its sandbox and got into Hugging Face's systems. Hugging Face's read of what it was after: the test solutions. It didn't want to solve the challenge. It wanted the answers.

Sixteen years. Same move. Don't do the work. Get the answers.

8. The answer key lives outside

07:30-08:30

Pause after "A second copy." before you explain it. Point at REJECTED, then ACCEPTED.

So door four closes differently. Not a bigger lock. The checker was already locked. A second copy.

The controller, the thing that decides, keeps its own copy of the answers. It runs the comparison outside the agent's folder. The agent's copy is a proposal. The controller's copy decides.

We did that. Same broken code: rejected. Fixed code: accepted, five of five.

The agent still writes the code. It can still propose tests. We didn't build the step that reviews those proposals. But the agent no longer grades itself.

That benchmark team did the same thing. Put the answers where the agent can't reach them.

9. How to check a door

08:30-09:40

Point at the four rows one at a time. Say the rule only after the fourth.

Every fix you just saw was recorded both ways. The cheat refused. The real job still working. We do that because our own test lied first.

Our release step stores one approved build, in one place, on a single-use approval. In our lab that place is a local database. We tested it. Wrong code: refused. Looked great.

Except our test sent the wrong code after the approval was already used up. That gate would have refused anything. Green, and it proved nothing.

We redid it on one approval. Wrong code: refused. Right code, wrong place: refused. Right code, right place: released. Same again: refused.

Now the no means something. The yes sits right next to it.

That's the check for every door. Try the bad thing. Then the good thing. Green checks lie. So do red ones.

10. Escaped without escaping

09:40-10:30

Say the title line, stop, then the two questions. Add nothing after "Thank you".

In our lab, the sandbox held every time. The checker held. Nothing broke out.

The agent left through the doors we opened for the job. A key that reached too far. A folder that took writes. A wire that reached another account. A verdict the agent could edit.

Your agent escaped without escaping the sandbox.

Monday, pick one door. Ask two things. What does it trust? Can the agent write that? Then try the bad thing, and try the good thing.

Every run is on GitHub, both ways, including the test that lied. Thank you.

Questions

11. What this experiment shows

Questions

Use only in questions.

We tested whether the setup allows these moves, and whether the fix stops them. We did not measure how often a model would try, and we did not run Opus 5.5; it's the example in the story. The five cases on the right are the whole test. Passing them proves those five cases and nothing more.

12. The key reached another team's data

Questions

Use only in questions.

A key limited to the task fixed it, and the task could still read its own input. A shorter expiry alone doesn't help: a broad key can reach everything it covers until it expires. The service has to check what the key is for.

13. The shared file changed on the host

Questions

Use only in questions.

The change stayed on the host, waiting for the next job. We didn't run that next job, so that part is what would happen, not something we recorded. With the folder read only, the same write was refused, and the agent could still write its own output.

14. The upload landed in another account

Questions

Use only in questions.

Reaching a service doesn't mean you may do anything there. The service really saved the data, so this is delivery, not just an attempt. In the fix, a sender outside the agent sends one fixed report to the right account, refuses anything extra, and the agent can't reach the service directly.

15. A bug in our old release step

Questions

Use only in questions.

Our old release step trusted an old fingerprint after the code changed. Triggering it needed direct access to our own controller, and we did not show the agent could get that. The new release step computes the fingerprint from the exact code it releases.

16. What we recorded

Questions

Use only in questions.

These count checks, not bugs found. The 49 are regression tests for our own code, not sandbox tests. The old bug is one reproduction of a mistake in our earlier release step.

17. Run it yourself

Questions

Use only in questions.

You need a throwaway Linux machine with bubblewrap, Python 3.10 or later, and make. On any computer, make test and make factory run the local checks, without a sandbox. GitHub Actions builds the PDFs on every change. No cloud account needed.

18. What the approval covers

Questions

Use only in questions.

The controller checks the results, then signs its decision together with the code, the answers and the destination, using a key only it holds. The approval names the task and the run, expires, and works once. The raw results themselves are not signed. Release in our lab means a row in a local database, not a deploy.

19. Sources

Questions

Use only in questions.

These are the sources for every real-world case in the talk. Each one is someone else's report. We don't claim their numbers as ours.